Week 1 Technical Design Document

MDM Address Validation & Enrichment

Group: 5010 - 5

Github: https://github.com/Aash27/mdm-address-validation

1. Project Context & Problem Statement

Honeywell maintains approximately 5 million Customer Master records in its MDM system. To keep these records useful for downstream business processes -billing, shipping, sales analytics, compliance - Honeywell relies on Dun & Bradstreet (D&B) to automatically enrich each record with firmographic data: the company's legal name, corporate hierarchy, industry codes, website, and other attributes.

However, approximately 76,000 records consistently fail D&B enrichment. D&B's matching algorithm requires a minimum level of data quality to identify a company -a recognizable name, a parseable address, a valid country. When the source data is too malformed, D&B simply returns nothing, leaving the record unenriched and degrading overall master data quality.

The root causes are not random. Through initial EDA on a 100-record sample, three distinct and recurring failure patterns were identified:

- Addresses stored in a non-standard pipe-delimited format (|#|#) that no external API can parse directly
- UTF-8 encoding corruption across multilingual records, turning é into Ã© and ü into Ã¼
- Records spanning multiple languages including Spanish, German, Catalan, and English with no language tagging

This pipeline focuses on two core capabilities: cleaning and validating addresses so they can be verified against a geolocation API, and enriching confirmed records with firmographic data from web sources. Pre-processing is a hard requirement before either step can function, since the APIs cannot handle raw pipe-delimited, encoding-corrupted input.

2. EDA Findings: What the Data Actually Looks Like

The EDA was run on a 100-record sample dataset. The purpose was to understand the real shape of the data before making any pipeline design decisions.

Output:

```
PS C:\Users\aashl\OneDrive\Desktop\mdm-address-validation> & C:/Users/aashl/AppData/Local/Programs/Python/Python310/python.exe c:/Users/aashl/OneDrive/Desktop/mdm-address-validation/eda.py

========= DATASET PREVIEW =========
    MDM_KEY ROWID_SYSTEM            SRC_KEY  ...   address_sub_category                                        name_reason                                        address_reason
0  20517805       BRP900         0002219388  ...  Full street addresses  The SOURCE_NAME 'Stone Security Llc' is a comp...  The address contains detailed street informati...
1   4546063       BRP900         0001325366  ...  Full street addresses  "Bimbo" is a well-known company name (Grupo Bi...  The address contains a detailed street number ...
2  13552380          HCE  0016S00003eiaYrQAI  ...  Full street addresses  "Laboratorio Milab" appears to be a company or...  The address contains a detailed street number ...
3  14080479      HBT_NEX  0011H00001v0m47QAA  ...  Full street addresses  The SOURCE_NAME 'American Industrial Sales - R...  The address contains a street-like component '...
4   6316350          EEP         0002009478  ...  Full street addresses  The SOURCE_NAME 'Cheniere Creole Trail Pipelin...  The address contains a detailed street number ...

[5 rows x 11 columns]

========= COLUMNS =========
Index(['MDM_KEY', 'ROWID_SYSTEM', 'SRC_KEY', 'SOURCE_NAME', 'FULL_ADDRESS',
       'name_category', 'name_sub_category', 'address_category',
       'address_sub_category', 'name_reason', 'address_reason'],
      dtype='object')
```

```
========== DATA INFO ==========
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 11 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   MDM_KEY               100 non-null    int64
 1   ROWID_SYSTEM          100 non-null    object
 2   SRC_KEY               100 non-null    object
 3   SOURCE_NAME           100 non-null    object
 4   FULL_ADDRESS          100 non-null    object
 5   name_category         100 non-null    object
 6   name_sub_category     100 non-null    object
 7   address_category      100 non-null    object
 8   address_sub_category  100 non-null    object
 9   name_reason           100 non-null    object
 10  address_reason        100 non-null    object
dtypes: int64(1), object(10)
memory usage: 8.7+ KB
```

Missing values: Zero across all columns in the sample. The data is structurally complete -every field exists -but the content inside those fields is where the quality breaks down.

Output:

```
========== MISSING VALUES ==========
MDM_KEY                0
ROWID_SYSTEM           0
SRC_KEY                0
SOURCE_NAME            0
FULL_ADDRESS           0
name_category          0
name_sub_category      0
address_category       0
address_sub_category   0
name_reason            0
address_reason         0
dtype: int64
```

2.1 Address Format Discovery

The single most important structural finding from EDA is that FULL_ADDRESS is not a plain address string. It uses a |#|# delimiter to pack multiple address components into one field:

5460 Whispering Oaks Ln|#|#Fort Worth|#|#TX|#|#US|#|#76108

This maps to:

Street |#|# City |#|# State |#|# Country |#|# Zip

This is critical because every external API -Google Places, Azure Maps -expects a clean human-readable address string. No API will understand |#|#. This means address parsing is not optional pre-processing, it is a hard requirement before any validation can happen.

Output:

```
========== SAMPLE FULL_ADDRESS VALUES ==========
0     5460 Whispering Oaks Ln|#|#Fort Worth|#|#TX|#|...
1     320-80 Avenue Se|#|#Calgary|#|#AB|#|#CA|#|#T2C...
2     Nueva Andres Bello 1940|#|#Independencia|#|#Re...
3               Waushara|#|#Berlin|#|#WI|#|#US|#|#54923
4     1970 Texas Eastern Rd|#|#Ragley|#|#LA|#|#US|#|...
5     122 E Cj Thomas Rd|#|#Monroe|#|#NC|#|#US|#|#28110
6            Reiherberg 112|#|#Celle|#|#|#|#DE|#|#29229
7     Calle JosÃ© Luis Cuevas 304|#|#Coatzacoalcos|#...
8      1200 Schafer St|#|#Bismarck|#|#ND|#|#US|#|#58506
9      1649 Horizon Pkwy|#|#Buford|#|#GA|#|#US|#|#30518
10    37 Temple Street|#|#Birmingham|#|#Birmingham|#...
11    2700 Miamisburg Centerville Rd|#|#Dayton|#|#OH...
12    LÃ¼becker Str. 122|#|#Magdeburg|#|#Sachsen-Anh...
13    Nave 2, Calle Ciro AlegrÃa 6|#|#MÃ¡laga|#|#MÃ...
14      200 Tyler Rd N|#|#Red Wing|#|#MN|#|#US|#|#55066
15    Carrer Santa Carolina 95 Bajos|#|#Barcelona|#|...
16    Polg.ind. San Cristobal, Calle Kripton 29|#|#V...
17    M-506 Km 20 5 Ctra. Villaviciosa, A Pinto|#|#F...
18    147 Queensway|#|#Sheerness|#|#KENT|#|#GB|#|#ME...
19    Carretera A La EstaciÃ³n Cortazar S/n|#|#Corta...
13    Nave 2, Calle Ciro AlegrÃa 6|#|#MÃ¡laga|#|#MÃ...
14      200 Tyler Rd N|#|#Red Wing|#|#MN|#|#US|#|#55066
15    Carrer Santa Carolina 95 Bajos|#|#Barcelona|#|...
16    Polg.ind. San Cristobal, Calle Kripton 29|#|#V...
13    Nave 2, Calle Ciro AlegrÃa 6|#|#MÃ¡laga|#|#MÃ...
14      200 Tyler Rd N|#|#Red Wing|#|#MN|#|#US|#|#55066
15    Carrer Santa Carolina 95 Bajos|#|#Barcelona|#|...
16    Polg.ind. San Cristobal, Calle Kripton 29|#|#V...
17    M-506 Km 20 5 Ctra. Villaviciosa, A Pinto|#|#F...
18    147 Queensway|#|#Sheerness|#|#KENT|#|#GB|#|#ME...
19    Carretera A La EstaciÃ³n Cortazar S/n|#|#Corta...
Name: FULL_ADDRESS, dtype: object
```

2.2 Encoding Corruption

Multilingual records -primarily Spanish, German, and Catalan -show consistent UTF-8 encoding corruption. The pattern is classic latin-1 / UTF-8 mismatch, where multi-byte characters were stored or exported incorrectly:

"BajÃ£Âo" → should be "Bajão"
"JosÃ©" → should be "José"
"MÃ¡laga" → should be "Málaga"

This affects both SOURCE_NAME and FULL_ADDRESS. If left uncorrected, these corrupted strings will cause address validation APIs to fail and LLM classification to be less accurate.

Output:

```
========== ENCODING ISSUES IN NAMES ==========
13    Nave 2, Calle Ciro AlegrÃa 6|#|#MÃ¡laga|#|#MÃ...
14      200 Tyler Rd N|#|#Red Wing|#|#MN|#|#US|#|#55066
15    Carrer Santa Carolina 95 Bajos|#|#Barcelona|#|...
16    Polg.ind. San Cristobal, Calle Kripton 29|#|#V...
17    M-506 Km 20 5 Ctra. Villaviciosa, A Pinto|#|#F...
18    147 Queensway|#|#Sheerness|#|#KENT|#|#GB|#|#ME...
19    Carretera A La EstaciÃ³n Cortazar S/n|#|#Corta...
Name: FULL_ADDRESS, dtype: object
```

2.3 Names Containing Numbers

17 out of 100 records contain numeric characters in SOURCE_NAME. These are not errors -they represent two legitimate patterns that need different handling:

- **Retail location codes:** Kaufland Markt 7540, Tj Maxx 01607 -a real company with a store number appended
- **Internal facility identifiers:** Everhome Suites, Hollyfrontier Corp with numeric suffixes -may indicate a branch or asset ID

The categorization agent must distinguish between these rather than treating all numeric names as invalid.

Output:

```
========== NAMES WITH NUMBERS ==========
17
```

2.4 Multilingual Records

The sample contains records from at least 5 countries with non-English address structures:

German: "Reiherberg 112, Celle, DE"
Spanish: "Carretera A La Estación Cortazar S/n"
Catalan: "Carrer Santa Carolina 95 Bajos, Barcelona"
Spanish: "Polg.ind. San Cristobal, Calle Kripton 29, Valencia"

Each language has its own address conventions, abbreviations, and street prefix formats. Calle, Carrer, Carretera all mean "street" but in different languages. A pre-processing step that only understands English abbreviations like St and Blvd would fail entirely on these records.

Output:

```
========== SAMPLE FULL_ADDRESS VALUES ==========
0     5460 Whispering Oaks Ln|#|#Fort Worth|#|#TX|#|...
1     320-80 Avenue Se|#|#Calgary|#|#AB|#|#CA|#|#T2C...
2     Nueva Andres Bello 1940|#|#Independencia|#|#Re...
3               Waushara|#|#Berlin|#|#WI|#|#US|#|#54923
4     1970 Texas Eastern Rd|#|#Ragley|#|#LA|#|#US|#|...
5     122 E Cj Thomas Rd|#|#Monroe|#|#NC|#|#US|#|#28110
6            Reiherberg 112|#|#Celle|#|#|#|#DE|#|#29229
7     Calle JosÃ© Luis Cuevas 304|#|#Coatzacoalcos|#...
8      1200 Schafer St|#|#Bismarck|#|#ND|#|#US|#|#58506
9      1649 Horizon Pkwy|#|#Buford|#|#GA|#|#US|#|#30518
10    37 Temple Street|#|#Birmingham|#|#Birmingham|#...
11    2700 Miamisburg Centerville Rd|#|#Dayton|#|#OH...
12    LÃ¼becker Str. 122|#|#Magdeburg|#|#Sachsen-Anh...
13    Nave 2, Calle Ciro AlegrÃa 6|#|#MÃ¡laga|#|#MÃ...
14      200 Tyler Rd N|#|#Red Wing|#|#MN|#|#US|#|#55066
15    Carrer Santa Carolina 95 Bajos|#|#Barcelona|#|...
16    Polg.ind. San Cristobal, Calle Kripton 29|#|#V...
17    M-506 Km 20 5 Ctra. Villaviciosa, A Pinto|#|#F...
18    147 Queensway|#|#Sheerness|#|#KENT|#|#GB|#|#ME...
19    Carretera A La EstaciÃ³n Cortazar S/n|#|#Corta...
13    Nave 2, Calle Ciro AlegrÃa 6|#|#MÃ¡laga|#|#MÃ...
14      200 Tyler Rd N|#|#Red Wing|#|#MN|#|#US|#|#55066
15    Carrer Santa Carolina 95 Bajos|#|#Barcelona|#|...
16    Polg.ind. San Cristobal, Calle Kripton 29|#|#V...
13    Nave 2, Calle Ciro AlegrÃa 6|#|#MÃ¡laga|#|#MÃ...
14      200 Tyler Rd N|#|#Red Wing|#|#MN|#|#US|#|#55066
15    Carrer Santa Carolina 95 Bajos|#|#Barcelona|#|...
16    Polg.ind. San Cristobal, Calle Kripton 29|#|#V...
17    M-506 Km 20 5 Ctra. Villaviciosa, A Pinto|#|#F...
18    147 Queensway|#|#Sheerness|#|#KENT|#|#GB|#|#ME...
19    Carretera A La EstaciÃ³n Cortazar S/n|#|#Corta...
Name: FULL_ADDRESS, dtype: object
```

2.5 Name Type Diversity

Even in 100 records the SOURCE_NAME field shows the full spectrum of pattern types the pipeline must handle:

"Stone Security Llc" → standard company
"Bimbo" → single word
"Spie Group Accounts Payable Somerset House"→ company + department + location
"American Industrial Sales - Recap W" → company + location descriptor
"Foot Action â Space524" → encoding issue + location code
"414 Las Cervezas Modelo Del Bajão" → number prefix + foreign language

Output:

```
========== SAMPLE SOURCE_NAME VALUES ==========
0                                   Stone Security Llc
1                                                Bimbo
2                                  Laboratorio Milab
3                  American Industrial Sales - Recap W
4                   Cheniere Creole Trail Pipeline, Lp
5                                     Willis Run Rate
6     Lower Saxony Academy For Fire And Disaster Pro...
7                       Us Fire Solutions, S.a. De C.v.
8                                   Hollyfrontier Corp
9                                     Everhome Suites
10          Spie Group Accounts Payable Somerset House
11                            Foot Action â Space524
12                                 Kaufland Markt 7540
13                          Nutrexpa Malaga (carreras)
14                                        Tj Maxx 01607
15                     Fund. Puigvert/i.u.n.a./hosp Sta
16                            Smntros. Garcichao, S.l.
17                                Hotel H2 Fuenlabrada
18                                         Hall And Kay
19      414 Las Cervezas Modelo Del BajÃ£Âo - Cortazar
Name: SOURCE_NAME, dtype: object
```

3. Technology Stack - Decisions & Justifications

3.1 Python 3.10

Why: Python is the industry standard for data engineering and AI pipelines. The entire ecosystem of relevant libraries -Pandas, OpenAI SDK, LangDetect -is Python-native. No other language offers this combination of data tooling and AI integration.

3.2 LLM - Azure OpenAI

Why: Azure OpenAI was selected as the LLM for the enrichment pipeline for three reasons. First, it operates within Honeywell's existing Azure infrastructure, keeping all data within their security boundary without requiring additional vendor onboarding. Second, it provides access to GPT-4o-level capability through a managed enterprise endpoint with reliable uptime and SLA guarantees. Third, it supports structured JSON output natively, which means the enrichment agent can return firmographic data in a machine-readable format without additional parsing logic.

3.3 Address Validation - Azure Maps API

Why: Azure Maps API was selected as the address validation tool because it integrates directly within Honeywell's existing Azure infrastructure, eliminating additional vendor procurement. It provides strong global address coverage, handles multilingual address formats, and returns standardized geocoded results including corrected street name, city, state, country, and zip. This is important given that the dataset contains records from the US, Germany, Spain, UK, Canada, and Chile.

3.4 Data Handling - Pandas

Why: Pandas is the standard Python library for tabular data manipulation. Reading CSVs, filtering records, adding columns, writing outputs -all of this is handled by Pandas with minimal code. For the POC scale of 7,600 records, Pandas is more than sufficient.

Future consideration: At the full 76K scale and certainly at 5M records, Pandas would need to be replaced with a distributed framework like Apache Spark or Polars. This is a known scaling consideration but out of scope for the POC.

3.5 Configuration & Security - python-dotenv

Why: API keys for Azure OpenAI, Azure Maps must never be hardcoded in the source code -this is a basic security requirement. python-dotenv loads these from a .env file that is excluded from version control via .gitignore. This is standard practice for any project that calls external APIs.

4. Enrichment Pipeline Design

Stage 4 is the core deliverable of this project. Its purpose is to take a record that has passed address validation and company verification, and return a fully populated set of firmographic attributes that can be written back to the MDM system.

4.1 What Enrichment Produces

For each successfully validated record, the enrichment stage attempts to populate the following fields:

Field	Description
Legal Company Name	The official registered name of the business
Website	Primary company URL
Industry Code	NAICS or SIC code where resolvable
Company Description	Short business description
Corporate Parent	Parent company name if the record is a subsidiary
Headquarters Address	Verified primary address of the company
Country of Operation	Primary country the business operates in

These fields mirror what D&B would have returned had the original record been clean enough to match. The goal is not to replace D&B permanently, but to produce enriched records that are now clean and structured enough to re-submit to D&B successfully in the future.

4.2 How Enrichment Works - The Azure OpenAI

The enrichment agent follows a three-step reasoning loop for each record:

Step 1 -Query Construction Azure OpenAI receives the cleaned SOURCE_NAME and validated address from the previous stages. It constructs one or more targeted search queries designed to find the company's official web presence. For example, for a record like "Stone Security LLC, Fort Worth, TX", the agent might generate queries such as "Stone Security LLC Fort Worth Texas official website" and "Stone Security LLC company profile".

Step 2 -Web Search via Azure OpenAI - The constructed queries are passed to the API, which returns structured Google Search results in JSON format -including page titles, URLs, and snippets. The agent receives back typically 5-10 results per query, covering the company's own website, LinkedIn profile, business directories, and news mentions where available.

Step 3 -Reasoning & Extraction via Azure OpenAI The search results are passed back to Azure OpenAI with a structured extraction prompt. The model reasons over the results and extracts the firmographic fields listed in Section 6.1. It is instructed to return only fields it can confirm from the search results, and to flag any field it cannot resolve with a null value rather than guessing. The output is a structured JSON object that maps directly to the MDM enrichment schema.

4.3 Enrichment Prompt Design

The prompt sent to Azure OpenAI for each enrichment call is structured as follows:

SYSTEM:

You are a company data enrichment agent for an MDM data quality pipeline at Honeywell.
You will be given a company name, a validated address, and a set of web search results.
Your job is to extract firmographic data about the company from the search results.
Only return fields you can confirm from the search results.
For any field you cannot confirm, return null.
Always return valid JSON. Never return plain text or explanations.

USER:

Company Name: {cleaned_source_name}
Validated Address: {validated_address}

Search Results:
{serper_results_json}

Return your enrichment output in this exact format:

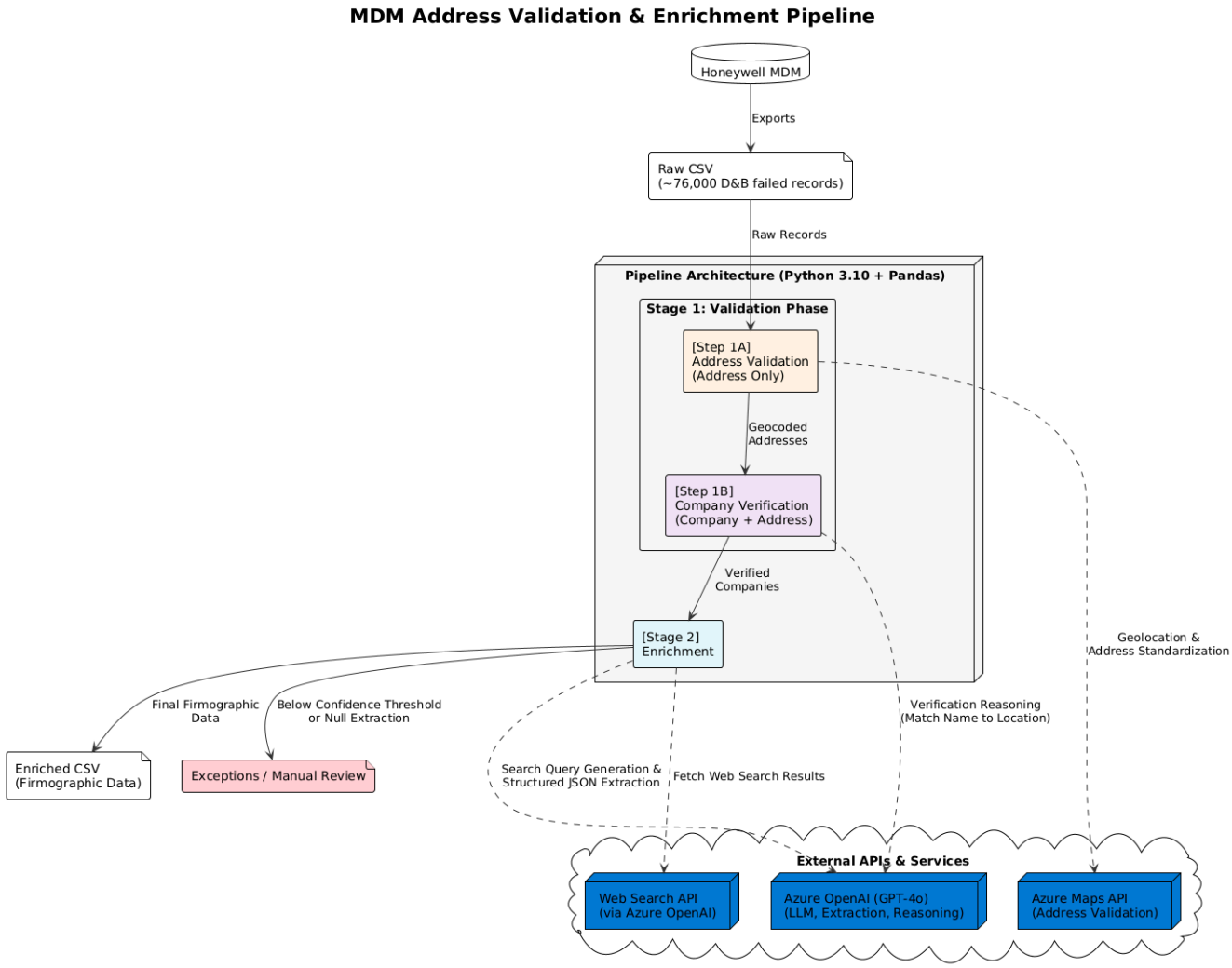
```
{
  "legal_name": "",
  "website": "",
  "industry_code": "",
  "company_description": "",
  "corporate_parent": "",
  "headquarters_address": "",
  "country_of_operation": "",
  "confidence_score": 0.0,
  "unresolved_fields": []

}
```

The confidence_score field (0.0 to 1.0) reflects how many of the target fields were successfully resolved. Records below a defined threshold (to be calibrated during POC) will be flagged for manual review rather than written back to the MDM system automatically.

4.4 Handling Enrichment Failures

Not every record will enrich successfully even after passing address validation. The following failure handling rules apply:

Failure Scenario	Handling
No search results returned	Flag record, log as "no web presence found", skip enrichment
Results found but no firmographic data extractable	Return partial JSON with null fields, flag for manual review
Confidence score below threshold	Write partial result to output, flag for human review queue
Azure OpenAI returns malformed JSON	Retry once with simplified prompt, log failure if retry fails

All failure outcomes are written to a separate exceptions log alongside the main enrichment output CSV, so the team can review failure patterns and refine the prompt or search query strategy across the 4 batches of POC processing.

5. Architecture Diagram